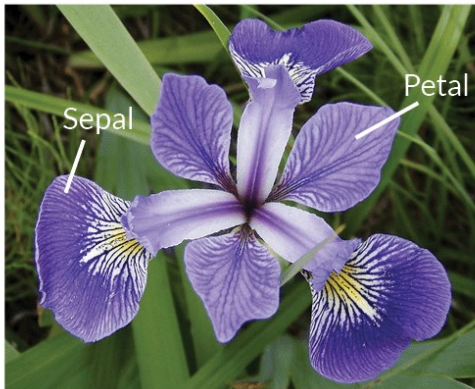


CLASSIFICATION

IRIS FLOWER



Iris Versicolor



Iris Setosa



Iris Virginica

Importing Libraries

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.preprocessing import LabelEncoder,
MinMaxScaler, StandardScaler
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
import seaborn as sns
```

```
/opt/conda/lib/python3.10/site-packages/scipy/__init__.py:146:
UserWarning: A NumPy version >=1.16.5 and <1.23.0 is required for this
version of SciPy (detected version 1.23.5
```

```
    warnings.warn(f"A NumPy version >={np_minversion} and
<{np_maxversion}")
```

```
df = pd.read_csv('../input/iris-flower-dataset/IRIS.csv')
df
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

```
[150 rows x 5 columns]
```

```
df.sample(5)
```

	sepal_length	sepal_width	petal_length	petal_width	species
141	6.9	3.1	5.1	2.3	Iris-virginica
46	5.1	3.8	1.6	0.2	Iris-setosa
113	5.7	2.5	5.0	2.0	Iris-virginica
84	5.4	3.0	4.5	1.5	Iris-versicolor
43	5.0	3.5	1.6	0.6	Iris-setosa

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df['species']=df['species'].str.replace('Iris-', '')
```

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

iris setosa



petal sepal

iris versicolor



petal sepal

iris virginica



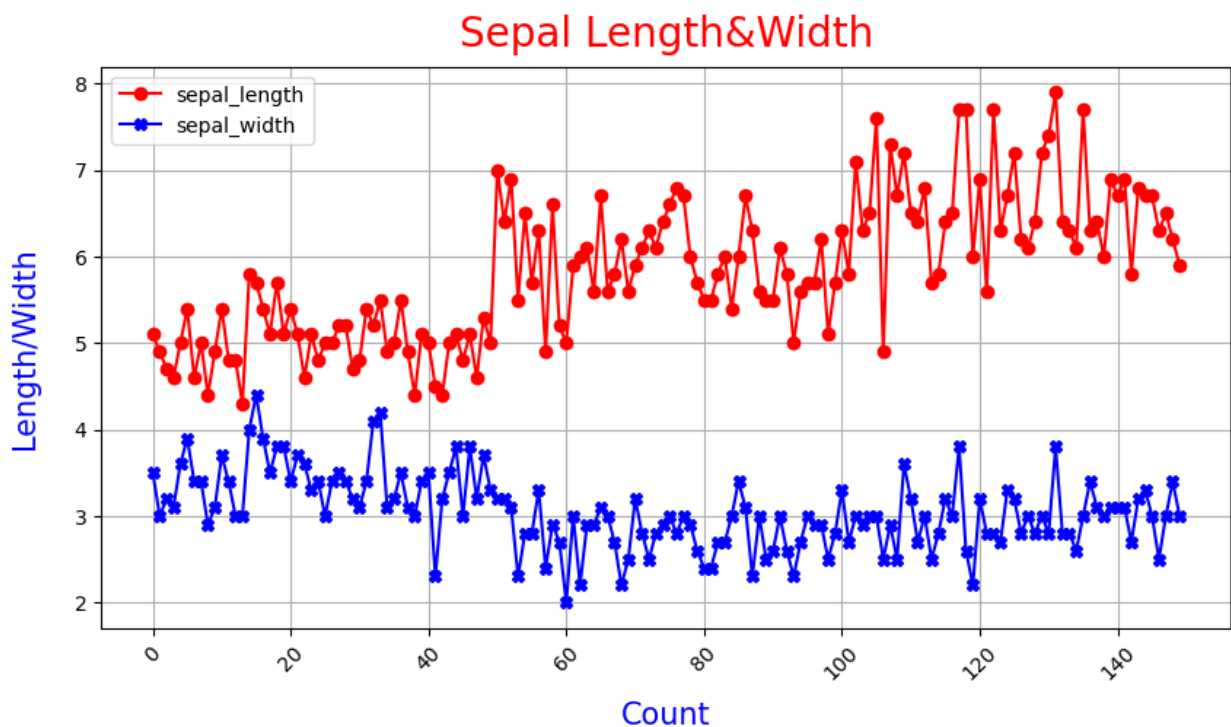
petal sepal

```
# df['sepal_length'].plot(kind='hist')
df['sepal_length'].plot(kind='line', legend=True, marker='o', color='r', fi
igsize=(10,5), grid=True, rot=45)
df['sepal_width'].plot(kind='line', legend=True, marker="X", color='b', fi
```

```

gsize=(10,5),grid=True,rot=45)
plt.title('Sepal
Length&Width',pad=10,loc='center',fontdict={'fontsize': 20, 'color':
'r','verticalalignment': 'baseline',})
plt.xlabel('Count',labelpad=20,loc='center',fontdict={'fontsize': 15,
'color': 'b','verticalalignment': 'baseline',})
plt.ylabel('Length/Width',labelpad=20,loc='center',fontdict={'fontsize
': 15, 'color': 'b','verticalalignment': 'baseline',})
Text(0, 0.5, 'Length/Width')

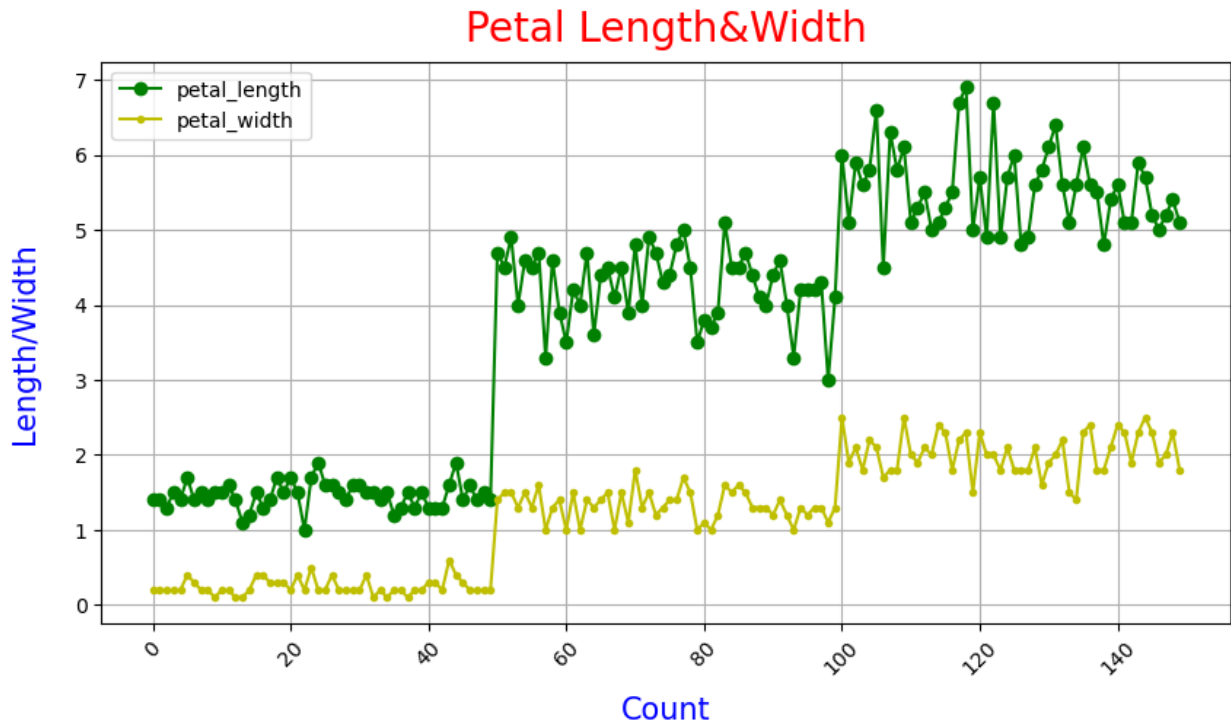
```



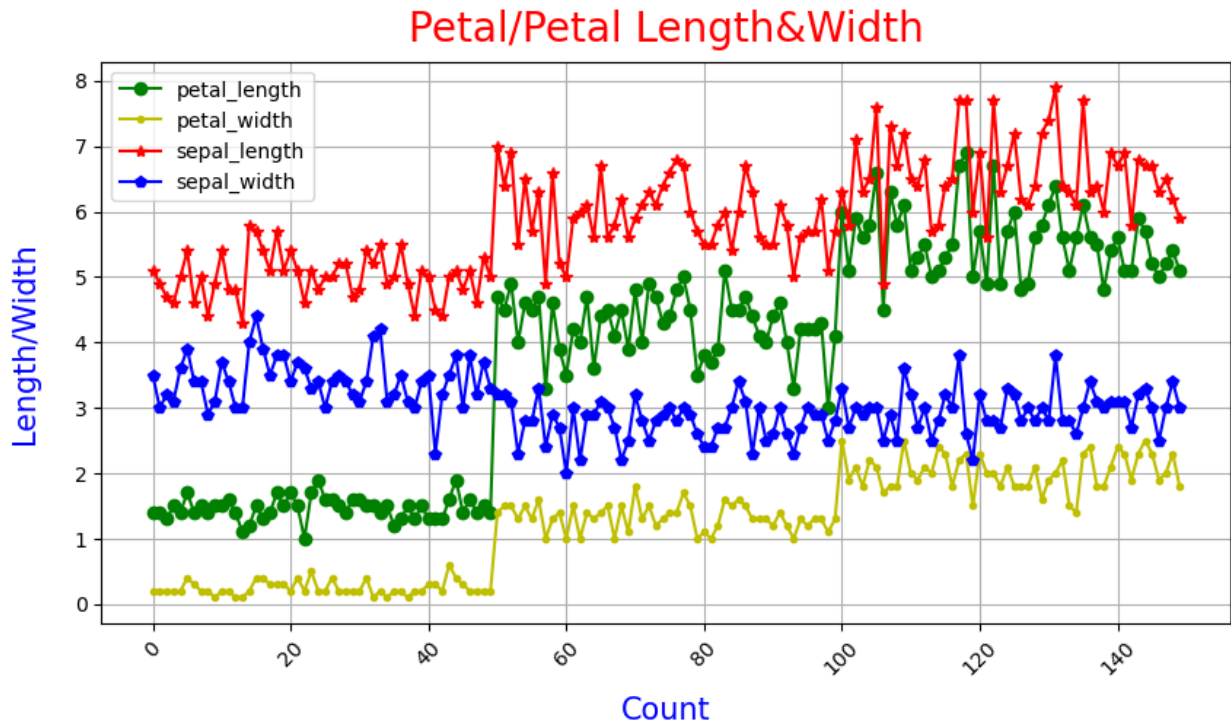
```

df['petal_length'].plot(kind='line',legend=True,marker='o',color='g',f
igsize=(10,5),grid=True,rot=45)
df['petal_width'].plot(kind='line',legend=True,marker=".",color='y',fi
gsize=(10,5),grid=True,rot=45)
plt.title('Petal
Length&Width',pad=10,loc='center',fontdict={'fontsize': 20, 'color':
'r','verticalalignment': 'baseline',})
plt.xlabel('Count',labelpad=20,loc='center',fontdict={'fontsize': 15,
'color': 'b','verticalalignment': 'baseline',})
plt.ylabel('Length/Width',labelpad=20,loc='center',fontdict={'fontsize
': 15, 'color': 'b','verticalalignment': 'baseline',})
Text(0, 0.5, 'Length/Width')

```



```
df['petal_length'].plot(kind='line',legend=True,marker='o',color='g',figsize=(10,5),grid=True,rot=45)
df['petal_width'].plot(kind='line',legend=True,marker=".",color='y',figsize=(10,5),grid=True,rot=45)
df['sepal_length'].plot(kind='line',legend=True,marker="*",color='r',figsize=(10,5),grid=True,rot=45)
df['sepal_width'].plot(kind='line',legend=True,marker="p",color='b',figsize=(10,5),grid=True,rot=45)
plt.title('Petal/Petal
Length&Width',pad=10,loc='center',fontdict={'fontsize': 20, 'color':
'r','verticalalignment': 'baseline',})
plt.xlabel('Count',labelpad=20,loc='center',fontdict={'fontsize': 15,
'color': 'b','verticalalignment': 'baseline',})
plt.ylabel('Length/Width',labelpad=20,loc='center',fontdict={'fontsize
': 15, 'color': 'b','verticalalignment': 'baseline',})
Text(0, 0.5, 'Length/Width')
```

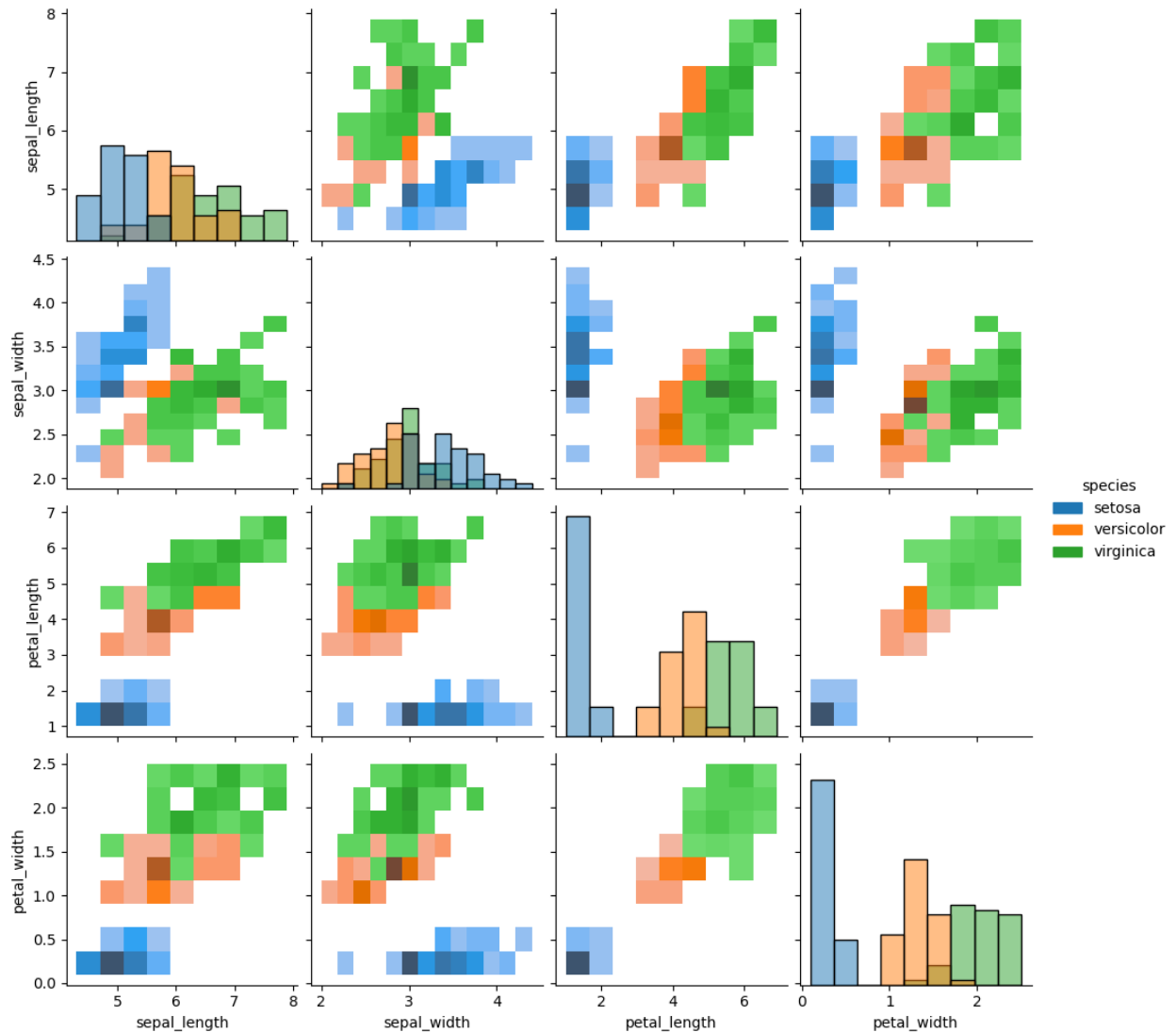


```
plt.figure(figsize=(17,6))
sns.pairplot(df,hue='species',kind='hist')

/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118:
UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)

<seaborn.axisgrid.PairGrid at 0x7da87ca66ec0>

<Figure size 1700x600 with 0 Axes>
```

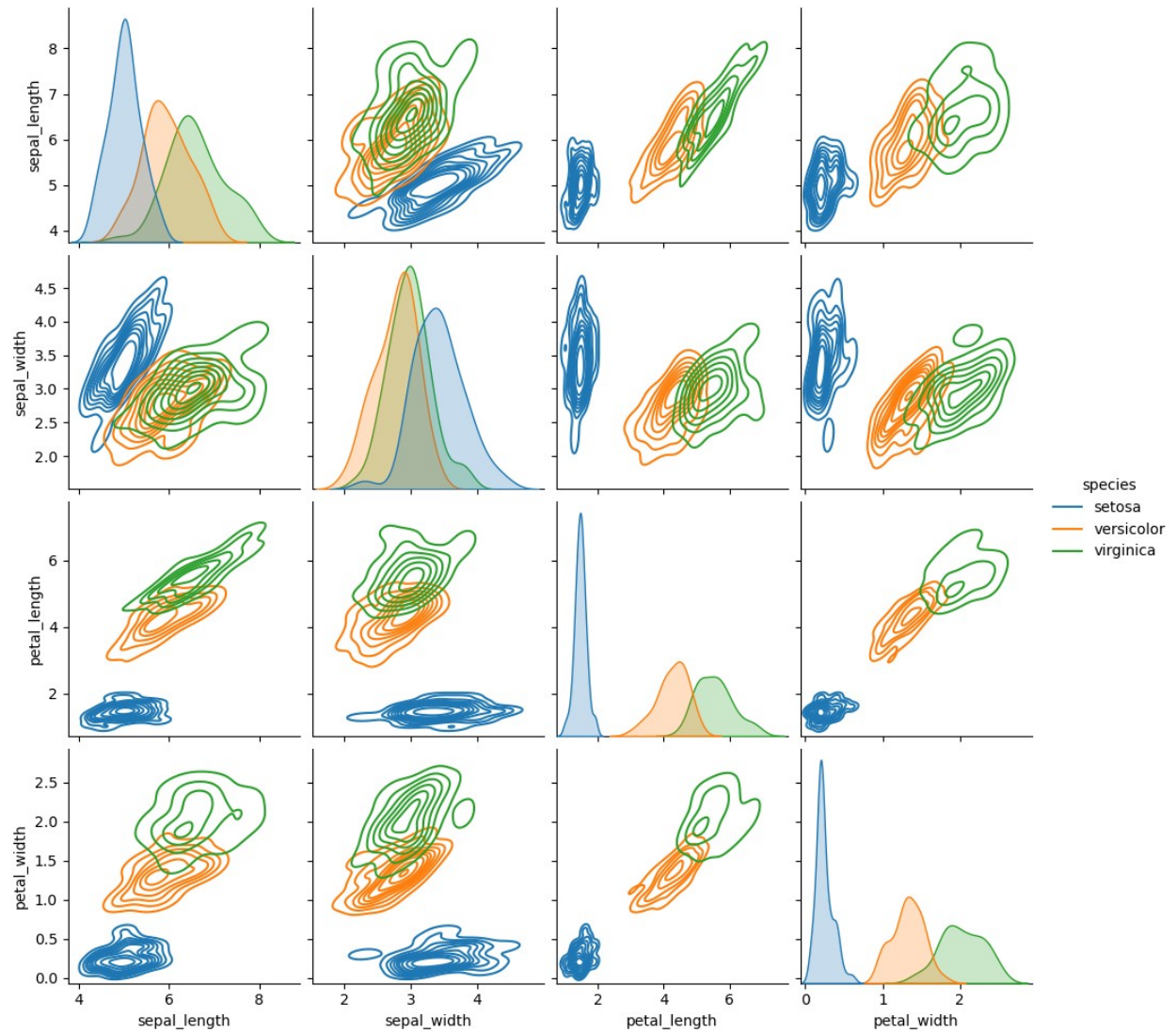


```
plt.figure(figsize=(17,6))
sns.pairplot(df,hue='species',kind='kde')

/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118:
UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)

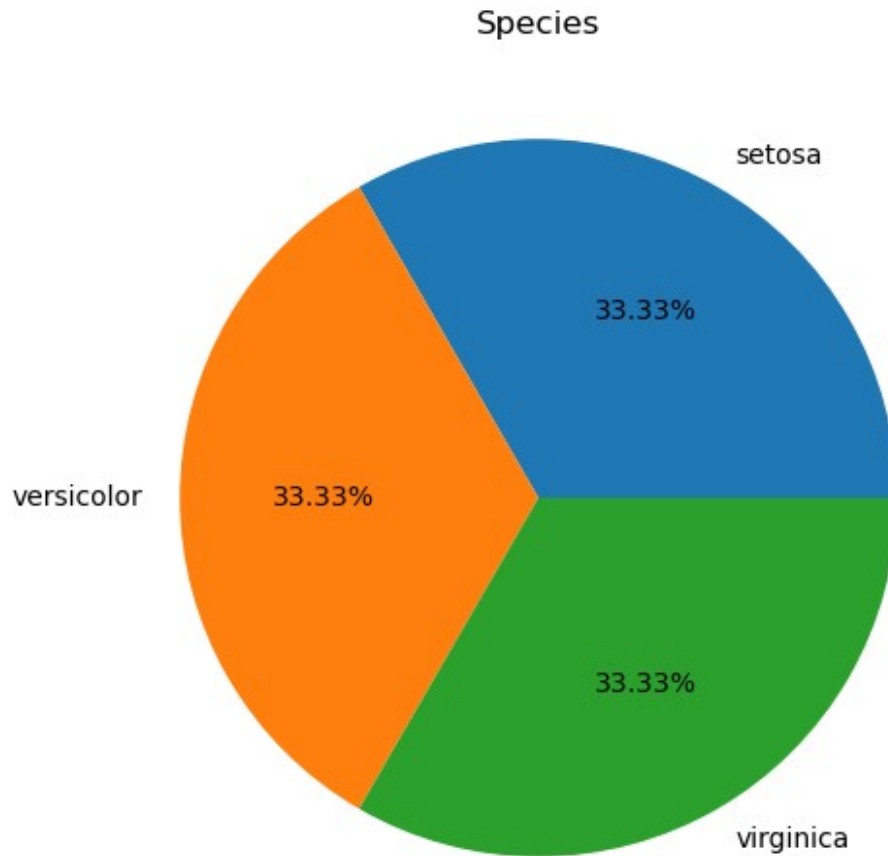
<seaborn.axisgrid.PairGrid at 0x7da878119870>

<Figure size 1700x600 with 0 Axes>
```

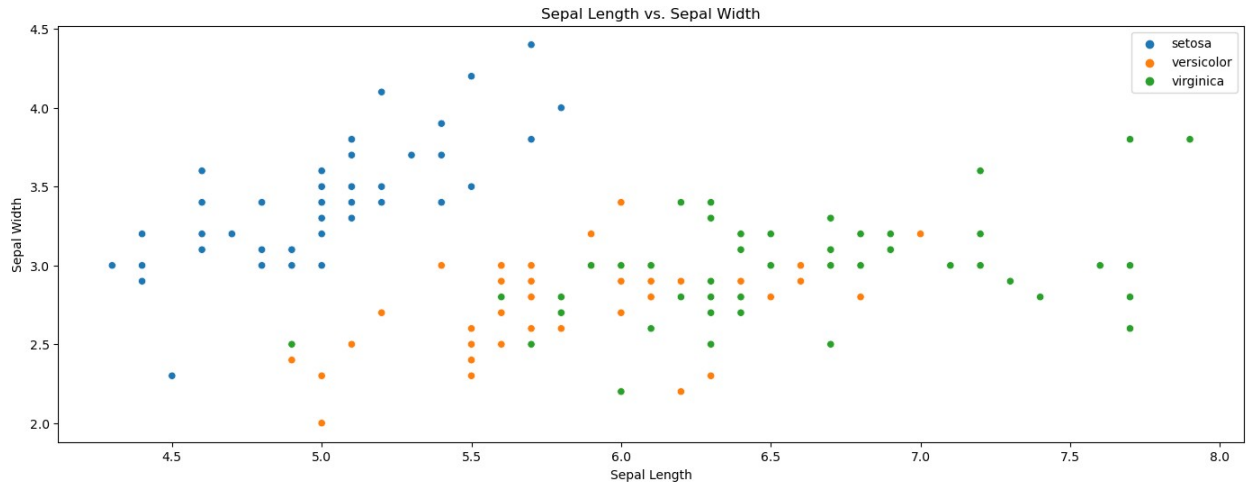



```
plt.figure(figsize=(17,6))
df['species'].value_counts().plot(kind='pie',autopct='%0.2f%
%',ylabel='',title='Species')
```

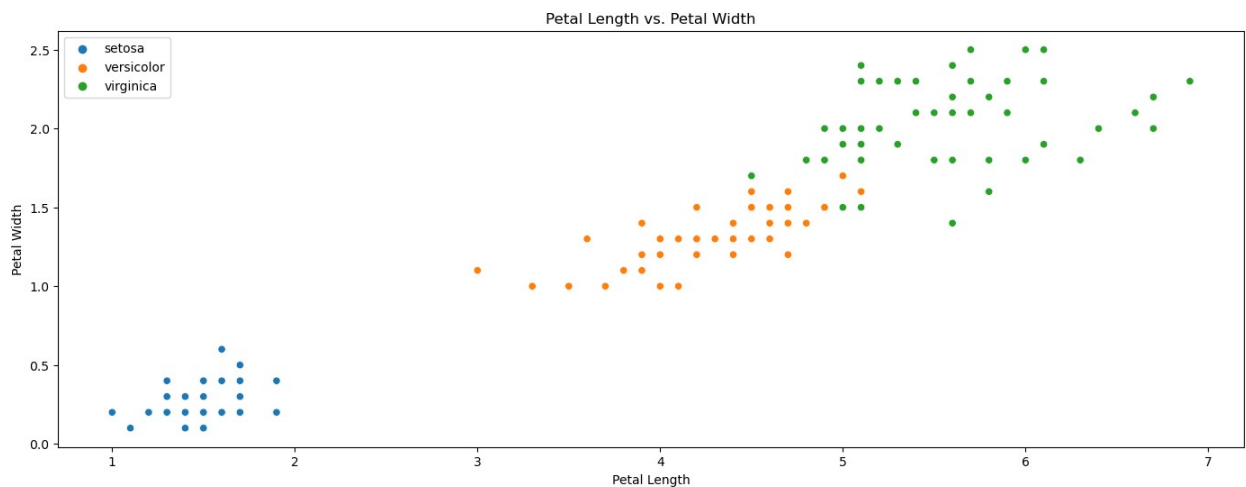
```
<Axes: title={'center': 'Species'}>
```

```
# # plt.figure(figsize=(17,6))
#
sns.scatterplot(df['sepal_length'],df['sepal_width'],hue=df['species']
)
# plt.legend(loc='upper right')
# plt.title('Sepal Length/Width')
plt.figure(figsize=(17, 6))
sns.scatterplot(data=df, x='sepal_length', y='sepal_width',
hue='species')
plt.legend(loc='upper right')
plt.title('Sepal Length vs. Sepal Width')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.show()
```



```
plt.figure(figsize=(17, 6))
sns.scatterplot(data=df, x='petal_length', y='petal_width',
hue='species')
plt.legend(loc='upper left')
plt.title('Petal Length vs. Petal Width')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.show()
```



```
# Create subplots for sepal width and sepal length
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Box plot for sepal width
sns.boxplot(data=df, x='species', y='sepal_width', ax=axes[0])
axes[0].set_title('Sepal Width by Species')
axes[0].set_xlabel('Species')
axes[0].set_ylabel('Sepal Width')

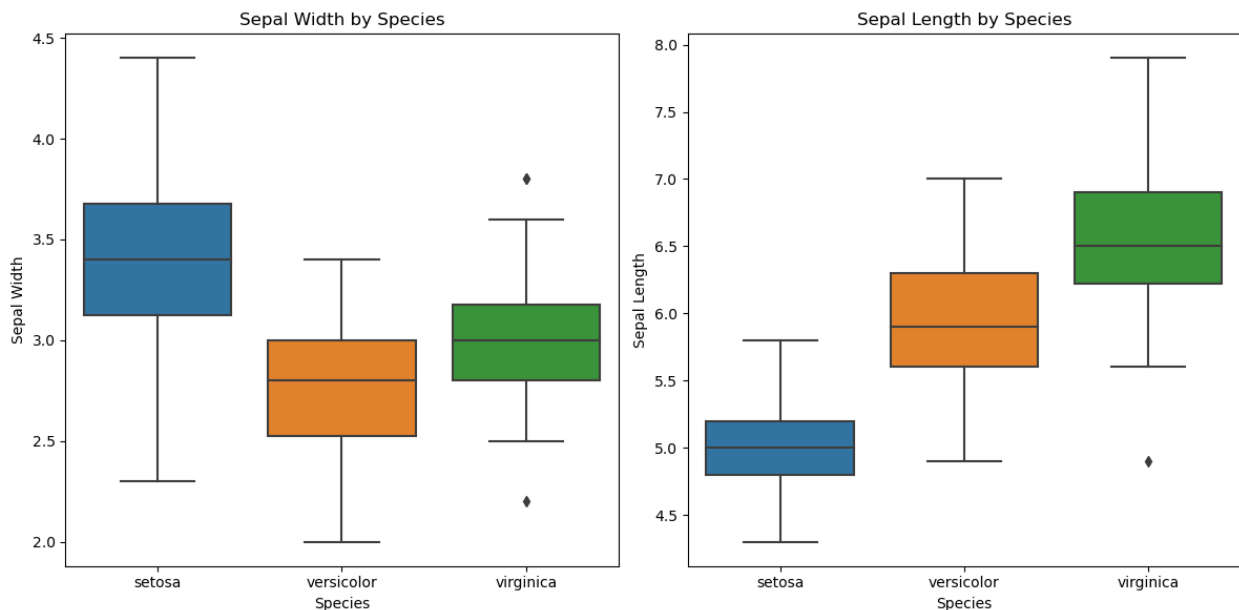
# Box plot for sepal length
```

```

sns.boxplot(data=df, x='species', y='sepal_length', ax=axes[1])
axes[1].set_title('Sepal Length by Species')
axes[1].set_xlabel('Species')
axes[1].set_ylabel('Sepal Length')

plt.tight_layout()
plt.show()

```



```

# # Density distribution plots for different features
# plt.figure(figsize=(10, 6))

# sns.distplot(df['sepal_length'], label='Sepal Length', hist=False)
# sns.distplot(df['sepal_width'], label='Sepal Width', hist=False)
# sns.distplot(df['petal_length'], label='Petal Length', hist=False)
# sns.distplot(df['petal_width'], label='Petal Width', hist=False)

# plt.legend()
# plt.title('Density Distribution of Iris Flower Features')
# plt.xlabel('Feature Values')
# plt.ylabel('Density')
# plt.show()
# Density distribution plots for different features
plt.figure(figsize=(10, 6))

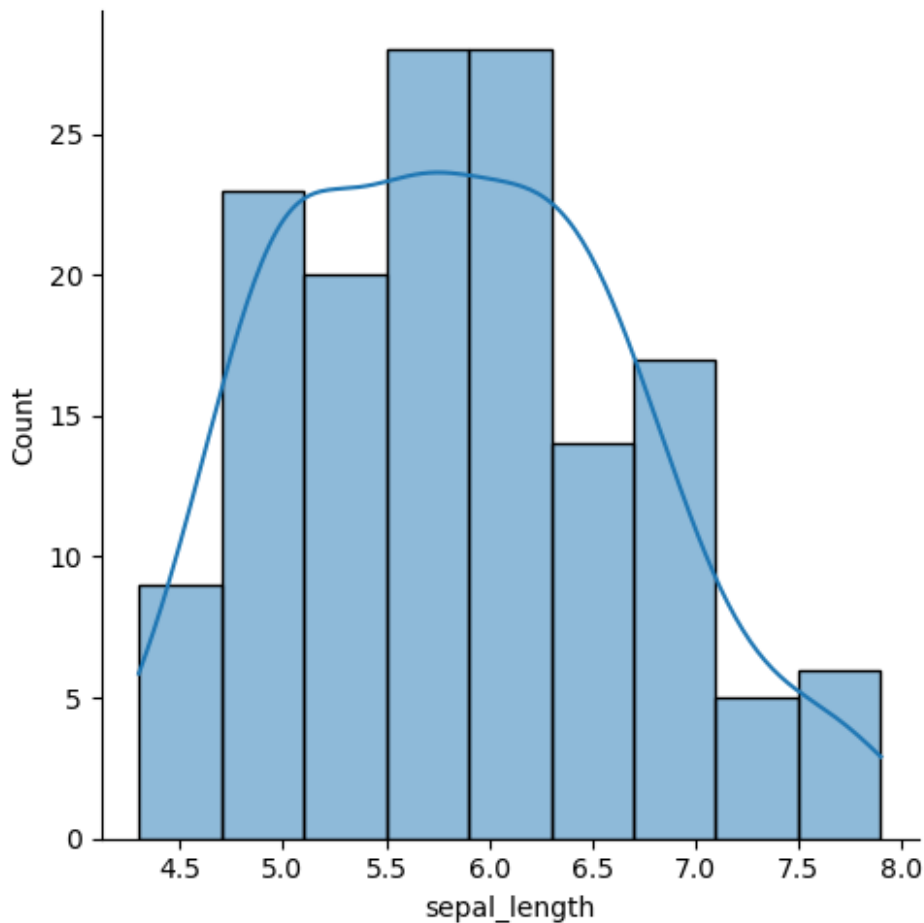
sns.displot(data=df, x='sepal_length', label='Sepal Length', kde=True)
sns.displot(data=df, x='sepal_width', label='Sepal Width', kde=True)
sns.displot(data=df, x='petal_length', label='Petal Length', kde=True)
sns.displot(data=df, x='petal_width', label='Petal Width', kde=True)

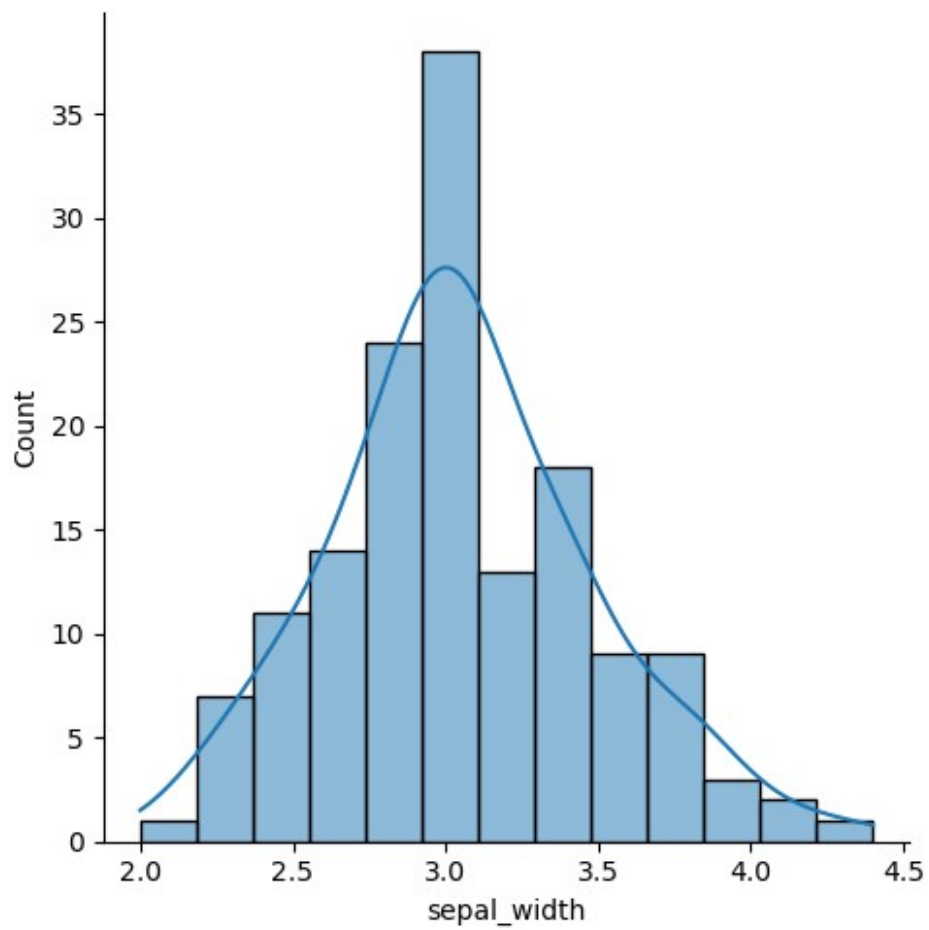
plt.legend()

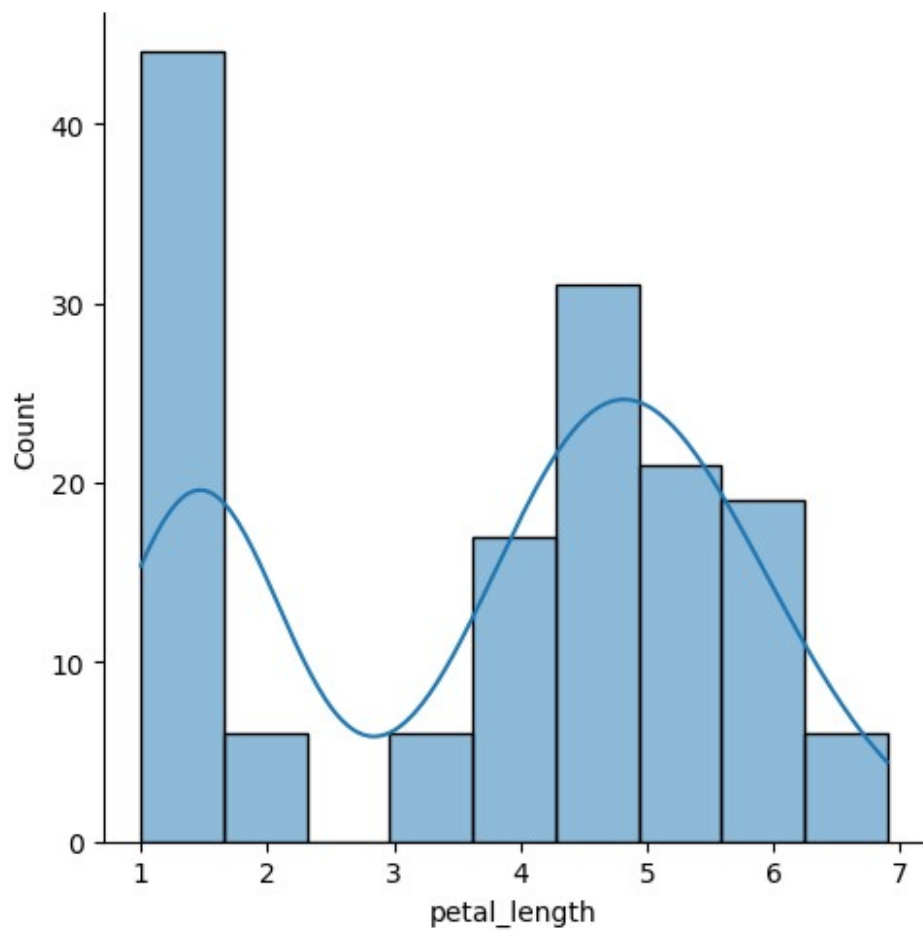
```

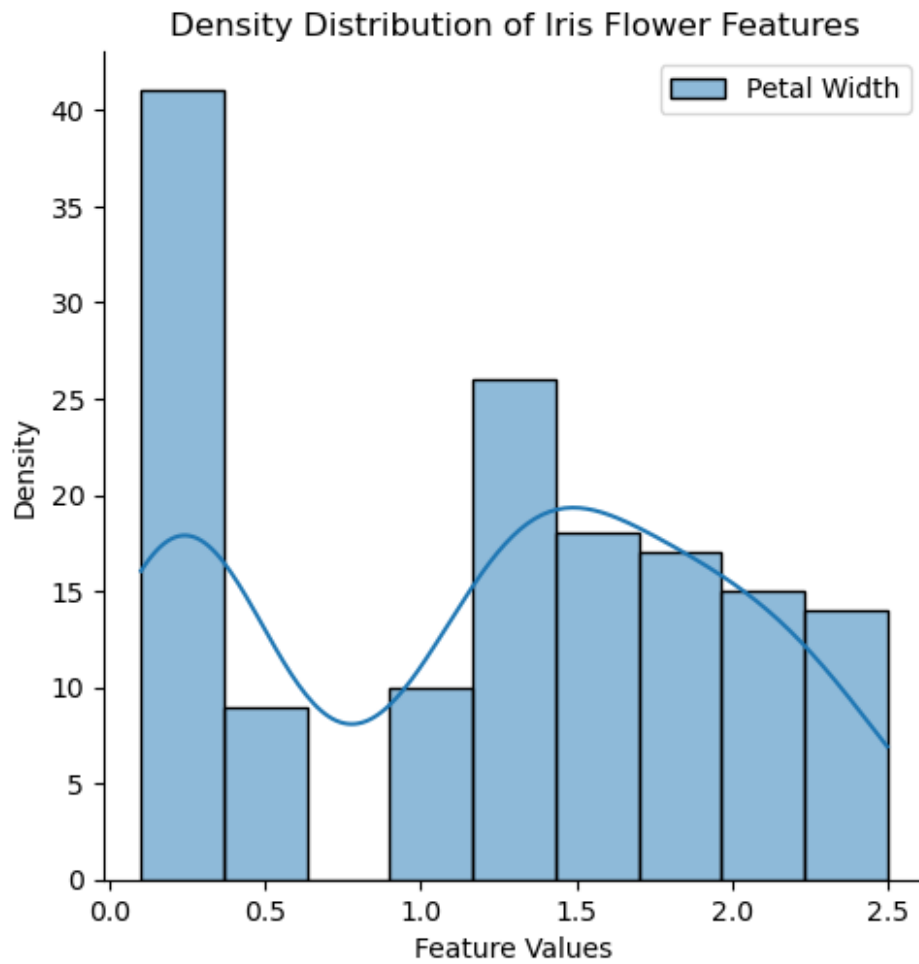
```
plt.title('Density Distribution of Iris Flower Features')
plt.xlabel('Feature Values')
plt.ylabel('Density')
plt.show()

/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118:
UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118:
UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118:
UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
/opt/conda/lib/python3.10/site-packages/seaborn/axisgrid.py:118:
UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
<Figure size 1000x600 with 0 Axes>
```









```
le = LabelEncoder()
```

```
df['species']=le.fit_transform(df['species'])
```

```
df
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

```
[150 rows x 5 columns]
```



```
X_train,X_test,y_train,y_test =
train_test_split(df.iloc[:, :4],df['species'],test_size=0.2,random_state=2)

lor = LogisticRegression()
lor.fit(X_train,y_train)
LogisticRegression()
test = lor.predict(X_test)
for tests in test:
    print("->",tests)

-> 0
-> 0
-> 2
-> 0
-> 0
-> 2
-> 0
-> 2
-> 2
-> 0
-> 0
-> 0
-> 0
-> 0
-> 1
-> 1
-> 0
-> 1
-> 2
-> 1
-> 2
-> 1
-> 2
-> 1
-> 1
-> 0
-> 0
-> 2
-> 0
-> 2

lor.score(X_test,y_test)
0.9666666666666667
y_test.values
```

```
array([0, 0, 2, 0, 0, 2, 0, 2, 2, 0, 0, 0, 0, 0, 1, 1, 0, 1, 2, 1, 1,
1,
      2, 1, 1, 0, 0, 2, 0, 2])
```

```
minmax = MinMaxScaler()
```

```
x_train= minmax.fit_transform(X_train)
```

```
x_test= minmax.fit_transform(X_test)
```

```
lor.fit(x_train,y_train)
```

```
LogisticRegression()
```

```
test1= lor.predict(x_test)
```

```
for test2 in test1:
```

```
    print("->",test2)
```

```
-> 0
```

```
-> 0
```

```
-> 2
```

```
-> 0
```

```
-> 0
```

```
-> 2
```

```
-> 0
```

```
-> 2
```

```
-> 2
```

```
-> 0
```

```
-> 0
```

```
-> 0
```

```
-> 0
```

```
-> 0
```

```
-> 2
```

```
-> 2
```

```
-> 0
```

```
-> 1
```

```
-> 2
```

```
-> 1
```

```
-> 2
```

```
-> 1
```

```
-> 2
```

```
-> 1
```

```
-> 1
```

```
-> 0
```

```
-> 0
```

```
-> 2
```

```
-> 0
```

```
-> 2
```

```
lor.score(x_test,y_test)
```

```
0.9
```

